

O objetivo desse exercício é implementar dois programas: um compactador e um descompactador de arquivos usando codificação de *Huffman*.

A árvore de *Huffman* pode ser representada em *Rust* usando o seguinte tipo de dados:

```
enum Huffman {
    Leaf {c:char, freq:u32},
    Node {freq:u32, left:Box<Huffman>, right:Box<Huffman>}
}
```

Sugestão para o formato do arquivo compactado:

u16		u32		u32	u32	u32	u32	...	u32	u32
n		t		c ₁	f ₁	c ₂	f ₂	...	c _n	f _n
b ₁	b ₂	b ₃	...						b _{t/8}	b _{t/8+1}

Sendo n o número símbolos (caracteres) distintos que ocorrem no arquivo original, t o total de caracteres do arquivo, c_i o i -ésimo símbolo do arquivo, f_i a frequência com que o símbolo c_i ocorre¹ no arquivo e b_s os bytes contendo os códigos de *Huffman*.

¹ A frequência de cada caractere é necessária para reconstruir a árvore de *Huffman*. Isso pode ser armazenado de forma mais eficiente usando código canônico de *Huffman*, mas para simplificar a implementação será aceito armazenar o símbolo e frequência